

RAG and Prompting Techniques

Dr. Adnan Arefeen

North South University

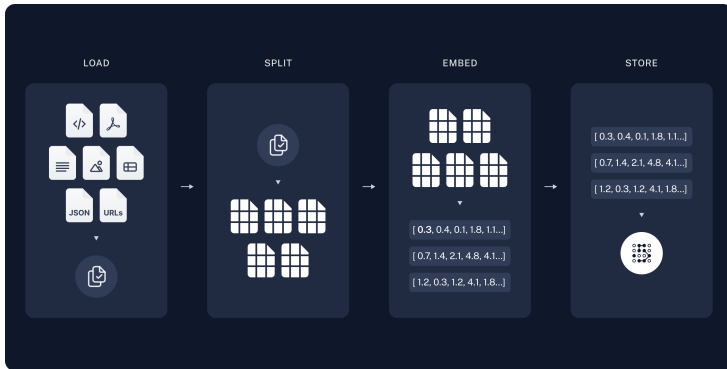
What is RAG?

- Retrieval Augmented Generation (RAG) = **Retrieval** (find relevant chunks) + **Generation** (LLM answers using retrieved context).
- Typical use: Q&A/chat over private corpora (docs, wikis, web pages).
- Two pipelines: **Indexing** (offline) and **Retrieval+Generation** (online).

High-Level Architecture

- ① **Load** documents (document loaders)
- ② **Split** into chunks (text splitters)
- ③ **Embed** chunks (embedding model)
- ④ **Store** in vector store (FAISS/Chroma/InMemory, etc.)
- ⑤ **Retrieve** top- k chunks per query
- ⑥ **Generate** answer with chat model/LLM using retrieved context

RAG components



1) Load

- Use *Document Loaders* to ingest sources (files, web pages, APIs).
- Example: WebBaseLoader to crawl a blog page; PDF/Text loaders for local files.

2) Split

- Apply *Text Splitters* (e.g., RecursiveCharacterTextSplitter).
- Choose chunk size/overlap to fit LLM context and preserve coherence.

3) Embed + Store

- Compute vector embeddings for each chunk.
- Insert into a **VectorStore** with metadata (source, section, etc.).
- Vector search enables fast similarity retrieval.

4) Retrieve

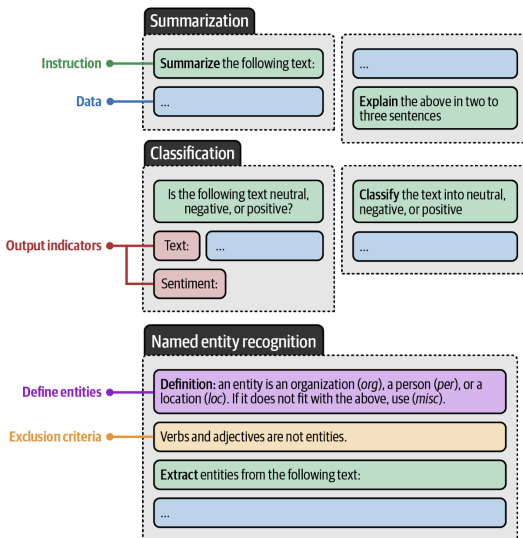
- Convert user question to an embedding; find top- k similar chunks.
- Optionally apply filters (metadata: date/section/type) or reranking.

5) Generate

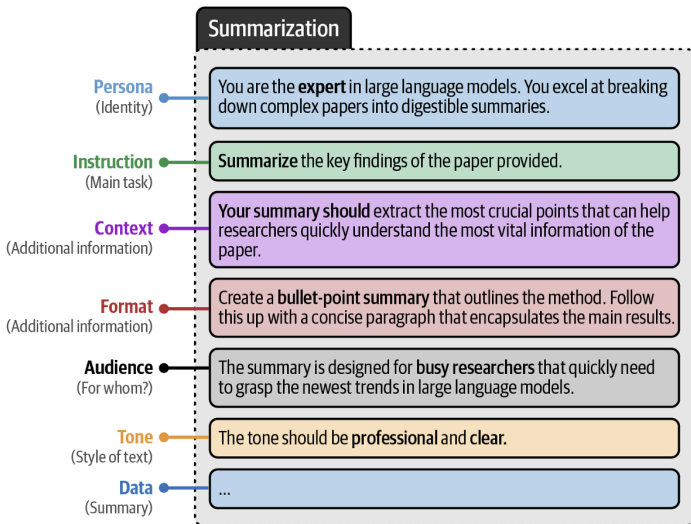
- Build a prompt template that injects *context* + *question*.
- Use a *ChatModel/LLM* to produce an answer grounded in retrieved chunks.
- Encourage citations / “I don’t know” behavior for safety.

Prompting Techniques

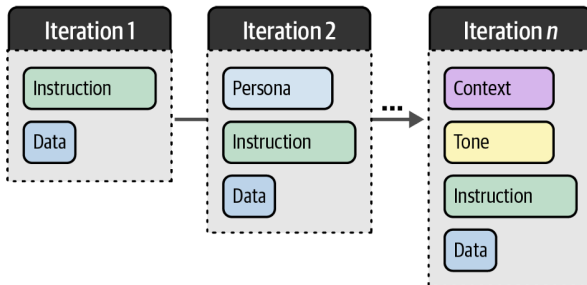
Task-aware simple prompting



Complex Reasoning



Iterative Prompting



Prompt Example

Prompt components

```
persona = "You are an expert in Large Language models. You excel at breaking  
down complex papers into digestible summaries.\n"  
instruction = "Summarize the key findings of the paper provided.\n"  
context = "Your summary should extract the most crucial points that can help  
researchers quickly understand the most vital information of the paper.\n"  
data_format = "Create a bullet-point summary that outlines the method. Follow  
this up with a concise paragraph that encapsulates the main results.\n"  
audience = "The summary is designed for busy researchers that quickly need to  
grasp the newest trends in Large Language Models.\n"  
tone = "The tone should be professional and clear.\n"  
text = "MY TEXT TO SUMMARIZE"  
data = f"Text to summarize: {text}"
```

*# The full prompt - remove and add pieces to view its impact on the generated
output*

```
query = persona + instruction + context + data_format + audience + tone + data
```

Prompt with or w/o examples

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

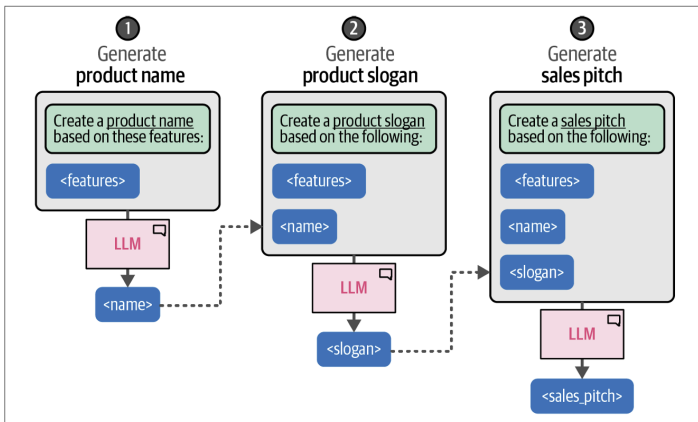
Text: I think the food was horrible...

Sentiment: **Negative**.

Text: I think the food was okay.

Sentiment:

Prompt with or w/o examples



Chain-of-Thought

Zero-shot chain-of-thought

Prompting without example

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Instruction

Let's think step-by-step.

Prime reasoning



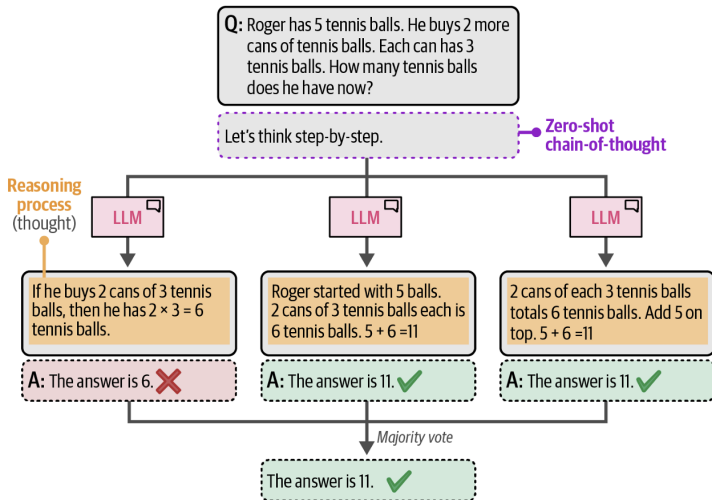
A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$.
The answer is 9. ✓

Reasoning process
(thought)

Self-consistency

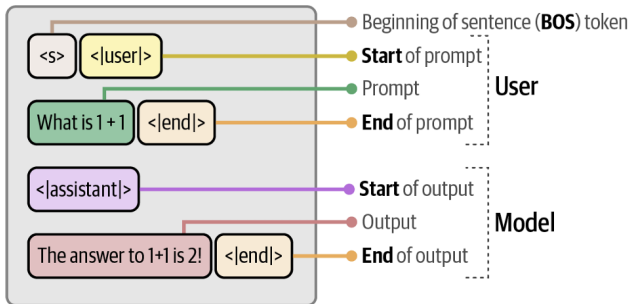
Self-consistency

Sampling from multiple paths



Phi-3 Prompt Instruction

Phi-3 template



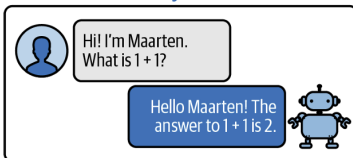
Phi-3 example Langchain

```
from langchain import PromptTemplate

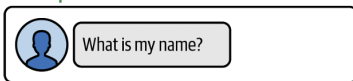
# Create a prompt template with the "input_prompt" variable
template = """<s><|user|>
{input_prompt}<|end|>
<|assistant|>"""
prompt = PromptTemplate(
    template=template,
    input_variables=["input_prompt"]
)
```

Conversation to Prompt

Conversation history



Prompt



Prompt template

